SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 1 – Research Paper Review / Literature Survey

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO5	**Assessment:**	Assessment Tool 1 – **Research Paper Review / Literature Survey**
Weightage:	20%	**Total Marks:**	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	Y. Ramadevi	**Reg. No.:**	192373065
Date:	21/08/2026	**Duration:**	60 minutes

Code of Conduct
*I **Y. Ramadevi** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*
Signature of the Student: **Y. Ramadevi**

Annexure A – Research Paper Review / Literature Survey

Sustainable DevOps and Green IT: Optimizing CI/CD Pipelines for Energy-Efficient Software Delivery

1. Introduction

Modern software delivery increasingly relies on continuous integration and continuous delivery (CI/CD), cloud infrastructure, containers, automated testing, and frequent deployments. These practices improve speed and reliability, but repeated builds, tests, scans, artifact creation, and infrastructure operations can increase computing energy use. Sustainable DevOps introduces environmental considerations into software engineering and operational decisions.

This literature survey reviews research on sustainable software engineering, energy-aware software, Green IT/AI, cloud energy efficiency, lifecycle sustainability, and environmental measurement. The findings are interpreted for the problem statement: “Sustainable DevOps and Green IT: Optimizing CI/CD Pipelines for Energy-Efficient Software Delivery.”

2. Objectives

Identify relevant scholarly research on energy-efficient and sustainable software engineering.

Review methods for measuring software energy consumption and carbon emissions.

Study optimization techniques applicable to CI/CD builds, tests, containers, infrastructure, and deployment.

Compare objectives, methodologies, tools, datasets/experimental setups, findings, and limitations.

Identify research gaps specifically related to sustainable CI/CD and DevOps.

Recommend practical future research directions.

3. Search Strategy and Selection Criteria

Aspect	Approach
Keywords	green software engineering; sustainable software; energy-efficient software; Green DevOps; green CI/CD; carbon-aware computing; software energy; sustainable cloud
Sources	Peer-reviewed journals, IEEE/ACM venues, ScienceDirect/Elsevier, Wiley, Springer, scholarly repositories
Selection	Relevance to software lifecycle, energy/carbon measurement, cloud/runtime efficiency, and practical optimization
Parameters	Objective, methodology, tools, dataset/experimental setup, metrics, findings, limitations, research gap

4. Comparative Literature Survey

Author/ Year	Research Problem	Objective	Methodology	Dataset/Tools/Setup	Key Finding	Limitation	Research Gap
Schwartz et al. (2019)	Green AI	Make efficiency a core evaluation criterion alongside accuracy.	Position paper / literature synthesis	Deep-learning computational cost and carbon concepts	Advocates reporting computational /financial cost and optimizing efficiency.	AI-centric; no concrete CI/CD implementation.	Translate efficiency metrics into automated DevOps quality gates.
Yokoyama et al. (2023)	Hardware and software	Study software,	Literature review +	XGBoost, Random Forest,	ARM and low-precision approaches	ML-focused; limited	Benchmark CI runners,

	aspects in ML energy consumption	hardware and energy-source effects on ML energy.	experiments	ARM, x86-64, GPU; multiple datasets	can reduce energy in studied workloads.	general CI/CD evidence.	build systems and test workloads using energy metrics.
Alizadeh & Castor (2024)	Green AI across runtime infrastructures	Measure energy effects of runtime infrastructure for DL models.	Empirical study	DL models, runtime infrastructures, target hardware	Runtime environment affects energy consumption.	Focused on DL execution rather than delivery pipelines.	Include runner/runtime choice in CI energy optimization.
Journal of Systems and Software (2024)	A Survey of Energy Concerns for Software Engineering	Map energy concerns across software engineering.	Broad literature survey across SWEBOK areas	Research literature across lifecycle areas	Identifies open challenges, trade-offs and practical green software approaches.	Broad survey; not a ready-made CI/CD framework.	Operationalize lifecycle energy practices inside CI/CD.
Information and Software Technology (2023)	Sustainable software engineering: Reflections on advances	Review sustainability concepts, measures, architectures and research trends.	Systematic mapping/reflection	234 identified studies; 102 relevant	Provides a roadmap and highlights emerging sustainability themes.	Sustainability remains broad and difficult to operationalize.	Connect sustainability metrics with release and DevOps decisions.
Lago et al.	Energy efficiency in cloud software architectures	Review cloud architectural approaches for energy efficiency.	Systematic literature review	Cloud software architectures	Architecture and resource decisions can affect energy use.	Limited direct treatment of automated CI/CD execution.	Combine cloud architecture optimization with pipeline scheduling.
Clemm et al. (2024)	Towards Green AI	Use lifecycle-based system	Lifecycle/system-thinking approach	Model, data, server and cloud	Environmental impact should be assessed across the	AI-oriented; no complete	Apply lifecycle boundaries to CI,

		thinking for environmental assessment.			software-hardware system.	DevOps implementation.	deployment and runtime.
Greening AI Research Agenda (2025)	Greening AI-enabled Systems with Software Engineering	Identify priority research challenges for sustainable AI-enabled systems.	Interdisciplinary workshop/research agenda	Energy assessment, standardization, benchmarking, architecture, runtime adaptation	Highlights standardization, benchmarking and runtime adaptation.	Research agenda rather than validated pipeline.	Develop standardized Green DevOps benchmarks and automated controls.

5. Individual Paper Reviews

5.1 Schwartz et al. (2019) – Green AI

The paper argues that computational efficiency should be evaluated alongside accuracy and other performance measures. It recommends reporting the cost of developing, training and running models. For Sustainable DevOps, this principle can be extended to pipeline duration, energy per run and carbon emissions. The major limitation is its AI-focused and conceptual nature; it does not specify automated CI/CD energy gates.

5.2 Yokoyama et al. (2023) – Hardware and Software Energy

The study considers software, hardware and carbon intensity as contributors to ML energy use. Experiments compare architectures and software techniques, including low precision. The key implication for DevOps is that runner architecture and software implementation affect energy consumption. The research is ML-specific, leaving conventional CI workloads underexplored.

5.3 Alizadeh & Castor (2024) – Runtime Infrastructure

This empirical study shows that runtime infrastructure can influence the energy consumption of deep-learning systems. The result supports evaluating not only application code but also the environment in which software executes. For CI/CD, runner type, container runtime and resource sizing should become sustainability considerations.

5.4 Energy Concerns for Software Engineering (2024)

This survey covers energy concerns across requirements, design, construction, testing, maintenance, configuration management, deployment, process and quality. It demonstrates that sustainability is a lifecycle-wide software engineering issue. The main gap is practical operationalization through automated CI/CD measurement and controls.

5.5 Sustainable Software Engineering (2023)

The review maps advances in sustainable software engineering and identifies measures, reference architectures and emerging research themes. It supports treating sustainability as a software quality and architecture concern. However, broad sustainability principles still require practical pipeline-level metrics and automation.

5.6 Cloud Software Architecture Energy Efficiency

The systematic review shows that cloud architecture and resource decisions can affect energy efficiency. This is relevant to CI/CD runners, staging environments and production infrastructure. The research gap is the lack of direct integration between cloud energy optimization and continuous delivery automation.

5.7 Towards Green AI (2024)

The lifecycle approach considers model, data, server and cloud together. This supports a wider sustainability boundary for DevOps that includes builds, tests, artifacts, deployment and runtime. The study does not provide a complete CI/CD implementation.

5.8 Greening AI Research Agenda (2025)

The research agenda identifies energy assessment, standardization, benchmarking, sustainability-aware architecture and runtime adaptation as important challenges. These priorities strongly support research into standardized Green DevOps measurements and automated sustainability controls.

6. Comparative Analysis

The literature shows a progression from conceptual efficiency principles to empirical energy measurement and lifecycle-wide sustainability assessment. Nevertheless, most studies focus on AI workloads, software architecture, or general software engineering rather than complete CI/CD pipelines.

Parameter	Literature Observation	Sustainable CI/CD Implication
Measurement	Energy, runtime, carbon intensity and computational cost are increasingly recognized.	Track energy/Wh and estimated CO2e per pipeline run and stage.
Optimization	Hardware, software, runtime and architecture all influence energy.	Use efficient runners, caching, incremental builds and resource right-sizing.
Scheduling	Carbon intensity is an important environmental factor.	Schedule flexible CI jobs when electricity is less carbon-intensive where feasible.
Lifecycle	Research increasingly considers the full software lifecycle.	Measure build, test, artifact, deployment, infrastructure and runtime stages.
Testing	Testing consumes resources but protects reliability.	Optimize test selection without compromising quality.
Standardization	Energy benchmarking remains an open challenge.	Define reproducible Green DevOps KPIs and thresholds.
Automation	Many recommendations are conceptual.	Make sustainability an automated CI/CD quality gate.

7. Limitations and Challenges

Energy measurement methods differ across studies, making direct comparison difficult.

Carbon emissions vary with electricity source, location, time and infrastructure.

Many studies focus on AI/ML workloads rather than ordinary CI/CD activities.

Standard CI platforms do not always expose reliable energy measurements.

Reducing runtime does not always reduce total environmental impact if resource intensity increases.

Reducing tests to save energy can increase defects and rework.

Cloud optimization must balance energy, cost, latency, availability and reliability.

Embodied impacts of servers and networking equipment are often separated from operational energy.

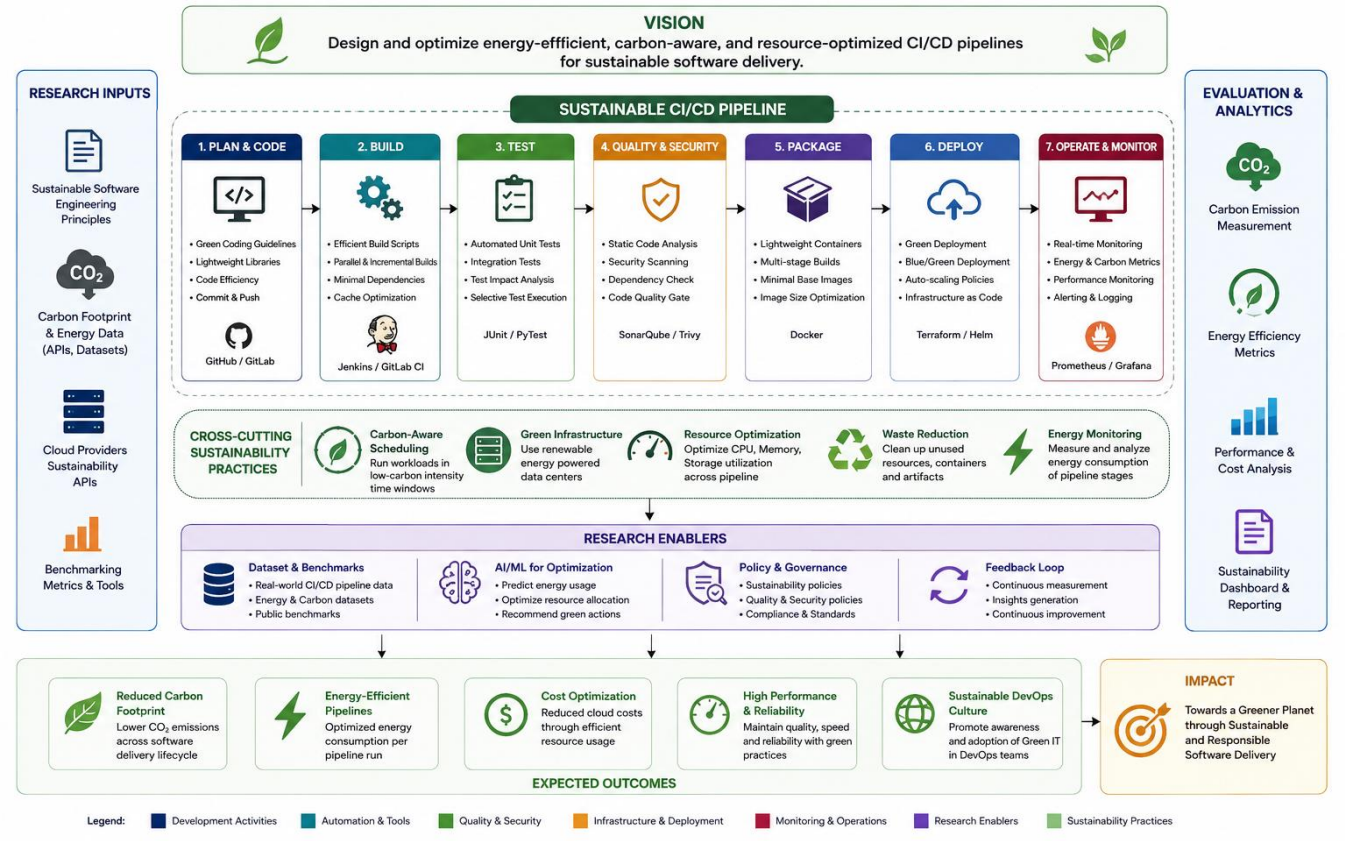
No universally adopted sustainability quality gate exists for mainstream CI/CD.

8. Identified Research Gap

The major gap is the lack of a practical, automated and standardized Green DevOps framework that directly integrates energy and carbon considerations into CI/CD decision-making. Existing research provides strong evidence for energy-aware software, efficient hardware, lifecycle thinking, cloud optimization and environmental measurement, but these ideas are rarely combined into a complete CI/CD workflow.

Gap	Importance	Proposed Response
No standard CI/CD energy metric	Pipelines cannot be compared consistently.	Define Energy per Successful Pipeline Run and Energy per Stage.
Limited carbon-aware scheduling	Flexible workloads may execute during high-carbon periods.	Use regional/time-based carbon intensity for scheduling.
Energy absent from quality gates	Pipelines optimize correctness/speed only.	Add optional energy and CO2e thresholds.
Limited CI runner comparison	Hardware changes energy/runtime behavior.	Benchmark ARM/x86/VM sizes for representative workloads.
Testing energy trade-off	More tests improve confidence but use compute.	Use selective, cached and incremental testing.
Poor end-to-end visibility	Build and runtime data are often separated.	Create sustainability dashboards covering CI/CD and runtime.

9. Proposed Sustainable CI/CD Research Framework



10. Future Research Directions

Develop an automated CI/CD energy meter for every pipeline stage.

Create carbon-aware scheduling for non-urgent CI workloads.

Introduce maximum energy or CO2e per successful build as a sustainability quality gate.

Use intelligent test selection to reduce redundant execution while preserving defect detection.

Benchmark ARM, x86 and other runner configurations using energy, time, cost and reliability.

Measure energy savings from dependency caching and incremental builds.

Create sustainability dashboards for energy, carbon, duration, resource utilization and deployment metrics.

Study multi-objective optimization of energy, cost, speed, reliability and test coverage.

Extend measurement to embodied carbon and hardware utilization.

Create reproducible Green DevOps benchmark workloads and datasets.

Integrate sustainability policies with Infrastructure as Code and autoscaling.

Develop organizational guidelines that treat sustainability as a software quality attribute.

11. Expected Contribution

A measurable framework for evaluating CI/CD environmental efficiency.

Automated energy and carbon indicators for pipeline stages.

Practical optimization methods that preserve quality and security.

Benchmarks for CI runner configurations and pipeline strategies.

Integration of sustainability metrics into DevOps dashboards.

Evidence-based trade-offs among delivery speed, reliability, cost and environmental impact.

12. Conclusion

The reviewed literature demonstrates that sustainable software engineering is becoming a measurable engineering discipline. Green AI research emphasizes efficiency and carbon reporting; software-engineering surveys extend energy concerns across the lifecycle; cloud architecture studies show the importance of infrastructure decisions; and lifecycle research emphasizes system-wide environmental assessment. These findings are directly relevant to sustainable CI/CD.

The strongest research opportunity is therefore to develop an energy- and carbon-aware CI/CD framework that measures pipeline stages, optimizes execution, and introduces sustainability as an explicit quality dimension. Such a framework can reduce unnecessary computing while preserving testing, security, reliability and fast software delivery.

13. References

Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2019). Green AI. arXiv:1907.10597. □cite□turn0academia35□

Yokoyama, A. M., Ferro, M., de Paula, F. B., Vieira, V. G., & Schulze, B. (2023). Investigating hardware and software aspects in the energy consumption of machine learning: A green AI-centric analysis. Concurrency and Computation: Practice and Experience, 35(24), e7825. □cite□turn0search2□

Alizadeh, N., & Castor, F. (2024). Green AI: A Preliminary Empirical Study on Energy Consumption in DL Models Across Different Runtime Infrastructures. CAIN 2024, 134–139. □cite□turn0search7□

A Survey of Energy Concerns for Software Engineering. (2024). Journal of Systems and Software, 210, 111944. □cite□turn0search6□

Sustainable software engineering: Reflections on advances in research and practice. (2023). Information and Software Technology, 164, 107316. □cite□turn0search10□

A systematic literature review on energy efficiency in cloud software architectures. Journal of Systems and Software. □cite□turn0search14□

Clemm, C., Stobbe, L., Wimalawarne, K., & Druschke, J. (2024). Towards Green AI: Current status and future research. arXiv:2407.10237. □cite□turn0academia36□

Greening AI-enabled Systems with Software Engineering: A Research Agenda for Environmentally Sustainable AI Practices. (2025). ACM SIGSOFT Software Engineering Notes. □cite□turn0search9□